



School of Computer Science

COMP47470: Big Data Programming

Neo4j NOSQL DBMS

Lab Practical 2

Module Coordinator	Ravi Reddy Manumachu
Email	ravi.manumachu@ucd.ie

Introduction: Installation, Service Checks	2
Neo4j StackOverflow	2
Cypher Terminology for Property Graph	3
Loading StackOverflow Property Graph	4
Queries to Understand Property Graph Structure	5
Basic Queries	9
Shortest Path Queries	13
Path-finding Queries to Solve	15

Introduction: Installation, Service Checks

Install Neo4j using the Installation and Quick Start guide in '**About the Tools**' in Brightspace. Before attempting the practical, set the password for the default user **neo4j**.

The guide's instructions are primarily intended for Ubuntu Linux. However, you are not required to use Unix/Linux platforms for the Neo4j practicals; Windows and Mac machines are also acceptable.

Ensure that you can start the Neo4j service and have access to tools like `neo4j-admin` and `neo4j-shell` to execute Neo4j commands.

Check the status of Neo4j service:

```
$ systemctl status neo4j
```

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ systemctl status neo4j
● neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-21 18:13:43 IST; 9min ago
     Main PID: 2219 (java)
       Tasks: 107 (limit: 38184)
      Memory: 1.0G (peak: 1.0G)
         CPU: 22.654s
       CGroup: /system.slice/neo4j.service
               └─2219 /usr/bin/java -Xmx128m -XX:IgnoreUnrecognizedVMOptions --sun-misc-unsafe-memory-access=allow -classpath "/usr/share/neo4j/lib/*:/usr/share/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/*" -XX:+UseG1GC
                 └─2685 /usr/lib/jvm/java-21-openjdk-amd64/bin/java -cp "/var/lib/neo4j/plugins/*:/etc/neo4j/*:/usr/share/neo4j/lib/*" -XX:+UseG1GC

Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.597+0000 INFO Anonymous Usage Data is being sent to Neo4j, see https://neo4j.com/usage
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.746+0000 INFO Bolt enabled on localhost:7687.
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.755+0000 INFO Fleet discovery broadcasts have been enabled for 3 networks: 172.17.0.0/16, 10.0.0.0/8, 192.168.0.0/24
Sep 21 18:13:46 system76-pc neo4j[2685]: 2026-09-21 17:13:46.981+0000 INFO This server has built-in support for online monitoring through Prometheus
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.141+0000 INFO HTTP enabled on localhost:7474.
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.142+0000 INFO Remote interface available at http://localhost:7474/
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO id: 048B00E6530EFB5746B7E505400EA773DB6ABF0CA6FC8495D883526A6D9
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO name: system
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO creationDate: 2026-07-22T08:53:49.604Z
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO Started.
```

You can start and stop the neo4j service as follows:

```
$ sudo systemctl start neo4j
```

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ systemctl status neo4j
○ neo4j.service - Neo4j Graph Database
   Loaded: loaded (/usr/lib/systemd/system/neo4j.service; enabled; preset: enabled)
   Active: inactive (dead) since Mon 2026-09-21 18:25:42 IST; 1min 1s ago
   Duration: 11min 48.349s
   Process: 2219 ExecStart=/usr/share/neo4j/bin/neo4j console (code=exited, status=0/SUCCESS)
   Main PID: 2219 (code=exited, status=0/SUCCESS)
   CPU: 27.938s

Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO name: system
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO creationDate: 2026-07-22T08:53:49.604Z
Sep 21 18:13:47 system76-pc neo4j[2685]: 2026-09-21 17:13:47.144+0000 INFO Started.
Sep 21 18:25:31 system76-pc systemd[1]: Stopping neo4j.service - Neo4j Graph Database...
Sep 21 18:25:31 system76-pc neo4j[2685]: 2026-09-21 17:25:31.985+0000 INFO Neo4j Server shutdown initiated by request
Sep 21 18:25:31 system76-pc neo4j[2685]: 2026-09-21 17:25:31.988+0000 INFO Stopping...
Sep 21 18:25:42 system76-pc neo4j[2685]: 2026-09-21 17:25:42.209+0000 INFO Stopped.
Sep 21 18:25:42 system76-pc systemd[1]: neo4j.service: Deactivated successfully.
Sep 21 18:25:42 system76-pc systemd[1]: Stopped neo4j.service - Neo4j Graph Database.
Sep 21 18:25:42 system76-pc systemd[1]: neo4j.service: Consumed 27.938s CPU time, 1.0G memory peak, 0B memory swap peak.
```

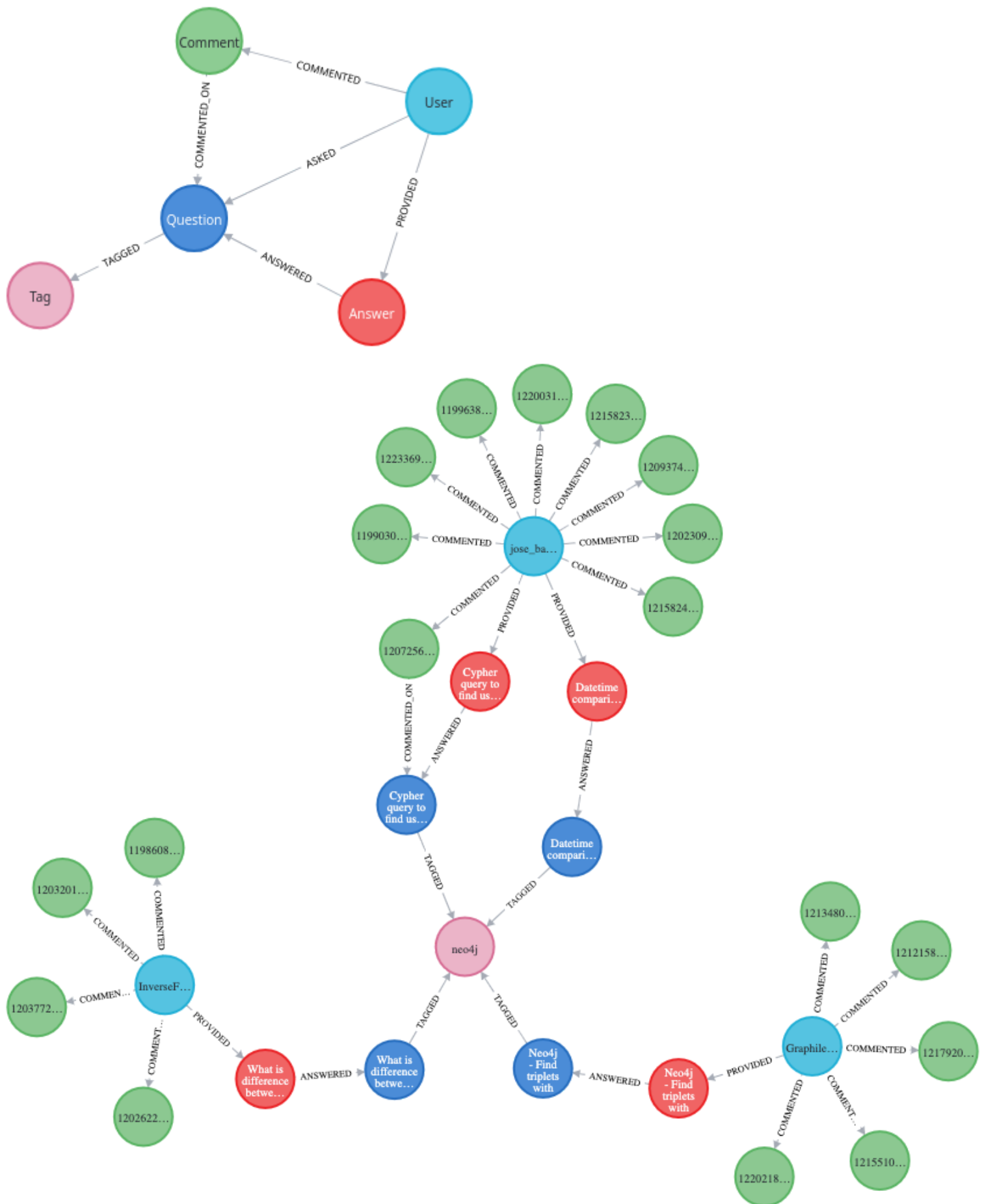
```
$ sudo systemctl stop neo4j
```

Let us start working with some Neo4j databases and run some graph queries.

You are provided a zip file containing the Neo4j database dumps. Untar the zip file.

Neo4j StackOverflow

We will load the **StackOverflow** property graph. It is a property graph including users, tags, and Q&A data retrieved from the website StackOverflow.



Cypher Terminology for Property Graph

The above property graph has **five** primary node types or labels. The labels are Question, User, Tag, Answer and Comment.

In this practical, we will work with two node labels and some relationships between them.

Each node type/label has specific properties:

Question has properties that include **uuid**, **title**, **creation date**, and **view_count**.

Answer has properties that include **uuid**, **title**, and **score**.

Six relationship types exist in this property graph. These are: ASKED, TAGGED, ANSWERED, PROVIDED, COMMENTED_ON, and COMMENTED.

The relationship types have no properties.

Loading StackOverflow Property Graph

But first, check the contents of the neo4j database. You must make sure it is empty before you attempt to run queries on the property graphs in this tutorial/practical.

\$ cypher-shell -u neo4j -p neo4j123 "SHOW DATABASES"

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ cypher-shell -u neo4j -p neo4j123 "SHOW DATABASES"
+-----+
| name      | type      | aliases | access      | address      | role      | writer | requestedStatus | currentStatus | statusMessage |
+-----+
| default   | home      | constituents |              |              |           |        |                |              |               |
+-----+
| "neo4j"   | "standard" | []      | "read-write" | "localhost:7687" | "primary" | TRUE   | "online"        | "online"      | ""            |
| "system"  | "system"   | []      | "read-write" | "localhost:7687" | "primary" | TRUE   | "online"        | "online"      | ""            |
+-----+
2 rows
ready to start consuming query after 411 ms, results consumed after another 47 ms
```

The **neo4j database** is your property graph, while **system** is a **metadata database** that cannot run graph queries.

The **neo4j database** is the default database where your actual graph data (nodes, relationships, properties) is stored.

```
manumachu@system76-pc:~/comp47470/SoftwarePackages$ cypher-shell -u neo4j -p neo4j123
Connected to Neo4j using Bolt protocol version 6.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> :use neo4j
neo4j@neo4j> MATCH (n) RETURN n LIMIT 10;
+-----+
| n      |
+-----+
| (:Station {name: "Peckham Rye"}) |
| (:Station {name: "Denmark Hill"}) |
| (:Station {name: "London Victoria"}) |
| (:Station {name: "Clapham High Street"}) |
| (:Station {name: "Elephant & Castle"}) |
| (:Stop {departs: 11:55Z}) |
| (:Stop {departs: 11:44Z}) |
| (:Stop {departs: 11:40Z}) |
| (:Stop {departs: 11:41Z}) |
| (:Stop {departs: 11:37Z}) |
+-----+
10 rows
ready to start consuming query after 18 ms, results consumed after another 2 ms
neo4j@neo4j> █
```

If it is not empty (as shown above for my local installation), you must delete the contents because the neo4j community edition allows working with just one property graph.

Either execute the following query in the neo4j shell:

neo4j@neo4j> MATCH (n) DETACH DELETE n;

0 rows

ready to start consuming query after 186 ms, results consumed after another 0 ms

Deleted 14 nodes, deleted 15 relationships

Or execute the following outside the neo4j shell:

cypher-shell -u neo4j -p neo4j123 "MATCH (n) DETACH DELETE n;"

You need to remove the following specific directories for the neo4j database. The exact paths depend on your Neo4j version and installation, but they are typically:

Databases: /var/lib/neo4j/data/databases/neo4j
Transactions: /var/lib/neo4j/data/transactions/neo4j

```
$ sudo systemctl stop neo4j
$ sudo rm -rf /var/lib/neo4j/data/databases/neo4j
$ sudo rm -rf /var/lib/neo4j/data/transactions/neo4j
```

Load the database using the **neo4j-admin** tool from the movies directory. Before that, you must stop the neo4j service.

```
$ cp Neo4j_LabPractical2_DBs/stackoverflow-50.dump /tmp/neo4j.dump
```

```
$ sudo -u neo4j neo4j-admin database load neo4j --from-path=/tmp --overwrite-destination=true
Done: 87 files, 6.705MiB processed in 0.090 seconds.
```

Start the neo4j service and run the queries in **cypher-shell**.

```
$ sudo systemctl start neo4j
$ cypher-shell -u neo4j -p neo4j123
```

Queries to Understand Property Graph Structure

Let us understand the structure of the graph.

1. List all node labels.

CALL db.labels();

```
neo4j@neo4j> CALL db.labels();
+-----+
| label |
+-----+
| "Question" |
| "User"      |
| "Tag"       |
| "Answer"    |
| "Comment"   |
+-----+

5 rows
ready to start consuming query after 63 ms, results consumed after another 5 ms
```

2. List all relationship types.

CALL db.relationshipTypes();

```
neo4j@neo4j> CALL db.relationshipTypes();
+-----+
| relationshipType |
+-----+
| "ASKED"          |
| "TAGGED"         |
| "ANSWERED"       |
| "PROVIDED"       |
| "COMMENTED_ON"  |
| "COMMENTED"     |
+-----+

6 rows
ready to start consuming query after 45 ms, results consumed after another 3 ms
```

3. List all property keys to see what attributes are stored on nodes and relationships.

CALL db.propertyKeys();

```
neo4j@neo4j> CALL db.propertyKeys();
+-----+
| propertyKey      |
+-----+
| "uuid"           |
| "title"          |
| "share_link"     |
| "favorite_count" |
| "creation_date"  |
| "display_name"   |
| "name"           |
| "accepted_answer_id" |
| "link"           |
| "view_count"     |
| "answer_count"   |
| "is_accepted"    |
| "body_markdown"  |
| "score"          |
+-----+

14 rows
ready to start consuming query after 42 ms, results consumed after another 2 ms
```

4. You can use the **db.schema.nodeTypeProperties()** procedure to retrieve the properties associated with each node label

CALL db.schema.nodeTypeProperties();

```
neo4j@neo4j> CALL db.schema.nodeTypeProperties();
+-----+-----+-----+-----+-----+
| nodeType | nodeLabels | propertyName | propertyTypes | mandatory |
+-----+-----+-----+-----+-----+
| "User"   | ["User"]  | "uuid"       | ["STRING NOT NULL", "INTEGER NOT NULL"] | TRUE      |
| "User"   | ["User"]  | "display_name" | ["STRING NOT NULL"] | TRUE      |
| "Tag"    | ["Tag"]   | "name"       | ["STRING NOT NULL"] | TRUE      |
| "Tag"    | ["Tag"]   | "link"       | ["STRING NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "uuid"       | ["INTEGER NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "title"      | ["STRING NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "link"       | ["STRING NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "is_accepted" | ["BOOLEAN NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "body_markdown" | ["STRING NOT NULL"] | TRUE      |
| "Answer" | ["Answer"] | "score"      | ["INTEGER NOT NULL"] | TRUE      |
| "Comment" | ["Comment"] | "uuid"       | ["INTEGER NOT NULL"] | TRUE      |
| "Comment" | ["Comment"] | "link"       | ["STRING NOT NULL"] | TRUE      |
| "Comment" | ["Comment"] | "score"      | ["INTEGER NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "uuid"       | ["INTEGER NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "title"      | ["STRING NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "creation_date" | ["INTEGER NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "accepted_answer_id" | ["INTEGER NOT NULL"] | FALSE     |
| "Question" | ["Question"] | "link"       | ["STRING NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "view_count" | ["INTEGER NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "answer_count" | ["INTEGER NOT NULL"] | TRUE      |
| "Question" | ["Question"] | "body_markdown" | ["STRING NOT NULL"] | TRUE      |
+-----+-----+-----+-----+-----+

21 rows
ready to start consuming query after 56 ms, results consumed after another 106 ms
```

- To find the properties associated with relationship types in Neo4j, you can use the built-in procedure **db.schema.relTypeProperties()**, which is the direct counterpart to the node procedure you already used.

CALL db.schema.relTypeProperties();

```
neo4j@neo4j> CALL db.schema.relTypeProperties();
+-----+
| relType          | propertyName | propertyTypes | mandatory |
+-----+
| ":\`ASKED\`"      | NULL        | NULL         | FALSE     |
| ":\`TAGGED\`"     | NULL        | NULL         | FALSE     |
| ":\`ANSWERED\`"   | NULL        | NULL         | FALSE     |
| ":\`PROVIDED\`"   | NULL        | NULL         | FALSE     |
| ":\`COMMENTED_ON\`" | NULL       | NULL         | FALSE     |
| ":\`COMMENTED\`"  | NULL        | NULL         | FALSE     |
+-----+

6 rows
ready to start consuming query after 54 ms, results consumed after another 27 ms
```

- Count nodes per label to see how many of each type of node exist.

MATCH (n)

RETURN labels(n) AS label, count(*) AS count

ORDER BY count DESC;

Let us understand the basic query above on finding nodes.

Nodes are referred to in Cypher using parentheses **(n)**. The **MATCH** clause is used to find a specific pattern in the graph, such as a specific node. The **RETURN** clause specifies which of the found graph patterns to return.

MATCH (n) Finds every node in the database. The **(n)** matches any node with no label or property filter.

RETURN labels(n) AS label For each node, extracts its label (e.g., Person, Movie) and renames the output column to label.

count(*) AS count How many nodes share the same label. The **count(*)** aggregates rows that have identical label values — this is the implicit grouping.

ORDER BY count DESC Sorts the result so the label with the most nodes appears at the top.

```
neo4j@neo4j> MATCH (n)
RETURN labels(n) AS label, count(*) AS count
ORDER BY count DESC;
+-----+
| label          | count |
+-----+
| ["Question"]  | 1589  |
| ["Comment"]   | 1396  |
| ["Answer"]    | 1367  |
| ["User"]      | 1365  |
| ["Tag"]       | 476   |
+-----+

5 rows
ready to start consuming query after 353 ms, results consumed after another 41 ms
```

- Count relationships per type to find out how many of each kind of relationship exist, revealing which connections dominate the graph.

MATCH ()-[r]->()
RETURN type(r) AS relationshipType, count(*) AS count
ORDER BY count DESC;

Let us figure out the basic query above on finding relationships.

To discover how nodes are connected to one another, relationships must be added to queries. Queries can specify relationship types, properties, and direction, as well as the start and end nodes of the pattern.

Relationships are represented in Cypher with arrows (e.g. ->) indicating the direction of a relationship.

MATCH ()-[r]->() Finds every relationship in the database. The empty parentheses () mean "any node, with no label or property filter" — so this matches any node on either end of the relationship.

RETURN type(r) AS relationshipType Extracts the relationship's type (e.g., ROUTE, OPERATES, ACTED_IN) and renames the output column to relationshipType.

count(*) AS count How many relationships share the same type. The implicit grouping is on the relationshipType value.

ORDER BY count DESC Sorts the result so the most frequent relationship type appears at the top.

```
neo4j@neo4j> MATCH ()-[r]->()
              RETURN type(r) AS relationshipType, count(*) AS count
              ORDER BY count DESC;
+-----+-----+
| relationshipType | count |
+-----+-----+
| "TAGGED"         | 4425  |
| "ASKED"          | 1589  |
| "COMMENTED_ON"   | 1396  |
| "COMMENTED"      | 1396  |
| "ANSWERED"       | 1367  |
| "PROVIDED"       | 1367  |
+-----+-----+
6 rows
ready to start consuming query after 165 ms, results consumed after another 32 ms
```

8. Show node labels and the relationship types connecting them.

MATCH (a)-[r]->(b)
RETURN DISTINCT labels(a) AS fromLabels,
type(r) AS relationship,
labels(b) AS toLabels
ORDER BY fromLabels, relationship, toLabels;

```
neo4j@neo4j> MATCH (a)-[r]->(b)
              RETURN DISTINCT labels(a) AS fromLabels,
              type(r) AS relationship,
              labels(b) AS toLabels
              ORDER BY fromLabels, relationship, toLabels;
+-----+-----+-----+
| fromLabels | relationship | toLabels |
+-----+-----+-----+
| ["Answer"] | "ANSWERED"   | ["Question"] |
| ["Comment"] | "COMMENTED_ON" | ["Question"] |
| ["Question"] | "TAGGED"     | ["Tag"] |
| ["User"] | "ASKED"      | ["Question"] |
| ["User"] | "COMMENTED"  | ["Comment"] |
| ["User"] | "PROVIDED"   | ["Answer"] |
+-----+-----+-----+
6 rows
ready to start consuming query after 78 ms, results consumed after another 32 ms
```

Basic Queries

1. Find the most viewed questions.

```
MATCH (q:Question)  
RETURN q.title AS question,  
    q.view_count AS views,  
    q.answer_count AS answers  
ORDER BY views DESC  
LIMIT 10;
```

MATCH (q:Question)

Finds all nodes labeled Question in the graph
Binds each one to the variable q

RETURN q.title AS question, q.view_count AS views, q.answer_count AS answers

Returns three properties from each matched node:

q.title → aliased as question
q.view_count → aliased as views
q.answer_count → aliased as answers

ORDER BY views DESC

Sorts by the views alias (i.e., q.view_count) in descending order — most-viewed first
Cypher allows referencing the RETURN alias directly in ORDER BY

LIMIT 10

Keeps only the top 10 rows after sorting

```
neo4j@neo4j> MATCH (q:Question)
RETURN q.title AS question,
       q.view_count AS views,
       q.answer_count AS answers
ORDER BY views DESC
LIMIT 10;
```

question	views	answers
"Java Input byte array has wrong 4-byte ending unit"	1851	0
"Given a plain text and a possible cipher text, determine whether the cipher text can be formed from the plain text using the mentioned scheme"	1820	3
"Database 'neo4j' is unavailable. Cannot reset neo4j database"	1291	2
"Neo.ClientError.Statement.SyntaxError: Invalid input 'l'; expected 'r/R' or 'o/O' (line 1, column 20 (offset: 19))"	1201	1
"Neo4j Filter function is no longer supported"	1072	1
"AES-128-CBC cipher in java"	1048	2
"Cypher: what is the use of YIELD?"	929	1
"Cypher commands not being recognised"	905	1
"How can I use the `with` statement with `call.apoc.do.when` in Neo4j?"	809	2
"Efficient way to import multiple csv's in neo4j"	698	2

10 rows
ready to start consuming query after 47 ms, results consumed after another 3 ms

2. Find the questions with the most answers.

```
MATCH (q:Question)  
RETURN q.title AS question,  
    q.answer_count AS answers  
ORDER BY answers DESC  
LIMIT 10;
```

MATCH (q:Question)

Finds all nodes labeled Question in the graph
Binds each one to the variable q

RETURN q.title AS question, q.answer_count AS answers

Returns two properties from each matched node:

q.title → aliased as question
q.answer_count → aliased as answers

ORDER BY answers DESC

Sorts the results by the answers alias (i.e., q.answer_count) in descending order - most answers first

Cypher allows you to reference the RETURN alias (answers) directly in ORDER BY, so you don't need to repeat q.answer_count

LIMIT 10

Keeps only the top 10 rows after sorting

```
neo4j@neo4j> MATCH (q:Question)
RETURN q.title AS question,
       q.answer_count AS answers
ORDER BY answers DESC
LIMIT 10;
```

question	answers
"Cannot merge the following node because of null property value for 'name':"	4
"Neo4J Cypher: Set property only if it is null but still return the node"	3
"How to convert a node to multiple relationships in Neo4j"	3
"filtering a fulltext search with a relationship constraint"	3
"Swapping elements in a list based on their match in a dictionary"	3
"Neo4j: Exclude certain nodes in variable path relationship"	3
"Cypher query to recursively traverse family tree?"	3
"Given a plain text and a possible cipher text, determine whether the cipher text can be formed from the plain text using the mentioned scheme"	3
"Order the result by the number of relationships"	3
"neo4j find all friends who posted something I liked"	3

10 rows
ready to start consuming query after 55 ms, results consumed after another 6 ms

3. Find the users who asked the most questions.

MATCH (u:User)-[:ASKED]->(q:Question)

RETURN u.display_name AS user,

count(q) AS questions_asked

ORDER BY questions_asked DESC

LIMIT 10;

```
neo4j@neo4j> MATCH (u:User)-[:ASKED]->(q:Question)
RETURN u.display_name AS user,
       count(q) AS questions_asked
ORDER BY questions_asked DESC
LIMIT 10;
```

user	questions_asked
"A. L"	13
"Matt Larson"	11
"Sama"	11
"sten"	11
"jos97"	10
"Aerodynamika"	8
"Simon"	8
"Ilnaz Nikseresht"	8
"marlon"	7
"ck22"	7

10 rows
ready to start consuming query after 58 ms, results consumed after another 10 ms

4. Find the most frequently used tags.

MATCH (q:Question)-[:TAGGED]->(t:Tag)

RETURN t.name AS tag,

count(q) AS questions

ORDER BY questions DESC

LIMIT 10;

```
neo4j@neo4j> MATCH (q:Question)-[:TAGGED]->(t:Tag)
              RETURN t.name AS tag,
                     count(q) AS questions
              ORDER BY questions DESC
              LIMIT 10;
```

tag	questions
"neo4j"	1524
"cypher"	1000
"neo4j-apoc"	122
"graph-databases"	108
"python"	83
"java"	77
"graph"	64
"database"	52
"spring-data-neo4j"	50
"spring-boot"	42

10 rows
ready to start consuming query after 90 ms, results consumed after another 37 ms

- Find the most viewed questions with a particular tag.

```
MATCH (q:Question)-[:TAGGED]->(t:Tag)
WHERE t.name = 'python'
RETURN q.title AS question,
       q.view_count AS views,
       q.answer_count AS answers
ORDER BY views DESC
LIMIT 10;
```

```
neo4j@neo4j> MATCH (q:Question)-[:TAGGED]->(t:Tag)
              WHERE t.name = 'python'
              RETURN q.title AS question,
                     q.view_count AS views,
                     q.answer_count AS answers
              ORDER BY views DESC
              LIMIT 10;
```

question	views	answers
"Given a plain text and a possible cipher text, determine whether the cipher text can be formed from the plain text using the mentioned scheme"	1820	3
"Neo4j cypher query results into Pandas DataFrame"	418	1
"Python: parallel Neo4j batch writes"	231	0
"How can I create GraphQL API with Django and Neo4j Database?"	194	2
"Which elements from networkx graph might become labels at neo4j graph?"	178	1
"simple example of working with neo4j python driver?"	166	1
"Get the nodes which are all connected with my input node using py2neo and flask"	165	1
"Neomodel: specify neo4j database name in connection string"	125	0
"where to find apoc.conf in neo4j sandbox"	108	1
"Neo4j, Python; OSError: [Errno 57] Socket is not connected"	104	0

10 rows
ready to start consuming query after 196 ms, results consumed after another 11 ms

- Find the users who provided the most answers.

```
MATCH (u:User)-[:PROVIDED]->(a:Answer)
RETURN u.display_name AS user,
       count(a) AS answers
ORDER BY answers DESC
LIMIT 10;
```

```
neo4j@neo4j> MATCH (u:User)-[:PROVIDED]->(a:Answer)
              RETURN u.display_name AS user,
                     count(a) AS answers
              ORDER BY answers DESC
              LIMIT 10;
```

user	answers
"cybersam"	211
"Graphileon"	120
"Tomaž Bratanič"	99
"jose_bacoy"	94
"InverseFalcon"	90
"Christophe Willemsen"	43
"fbiville"	32
"Luanne"	27
"Rajendra Kadam"	18
"Brakebein"	17

10 rows
ready to start consuming query after 51 ms, results consumed after another 7 ms

- Find users who answered the most Python questions.

```

MATCH (u:User)-[:PROVIDED]->(a:Answer)-[:ANSWERED]->(q:Question)-[:TAGGED]->(t:Tag)
WHERE toLower(t.name) = 'python'
RETURN u.display_name AS user,
       count(DISTINCT q) AS questions_answered
ORDER BY questions_answered DESC
LIMIT 10;

```

```

MATCH (u:User)-[:PROVIDED]->(a:Answer)-[:ANSWERED]->(q:Question)-[:TAGGED]->(t:Tag)

```

Traverses a four-hop path through the graph:

- (u:User) — a user node
- [:PROVIDED]-> — that user provided an Answer
- [:ANSWERED]-> — the answer addresses a Question
- [:TAGGED]-> — the question is tagged with a Tag

Each matched row is one (user, answer, question, tag) combination

```

WHERE toLower(t.name) = 'python'

```

Filters to only tag nodes whose name is "python" (case-insensitive)

toLower() normalizes the stored value so "Python", "PYTHON", and "python" all match

```

RETURN u.display_name AS user, count(DISTINCT q) AS questions_answered

```

Returns each user's display name

count(DISTINCT q) counts the unique questions that user has answered

DISTINCT is important because a user could have posted multiple answers to the same question - without DISTINCT, that question would be counted more than once

```

ORDER BY questions_answered DESC

```

Sorts users by their unique-question count, highest first

References the RETURN alias directly

```

LIMIT 10

```

Keeps only the top 10 users

```

neo4j@neo4j> MATCH (u:User)-[:PROVIDED]->(a:Answer)-[:ANSWERED]->(q:Question)-[:TAGGED]->(t:Tag)
WHERE toLower(t.name) = 'python'
RETURN u.display_name AS user,
       count(DISTINCT q) AS questions_answered
ORDER BY questions_answered DESC
LIMIT 10;

```

user	questions_answered
"jose_bacoy"	7
"Nigel Small"	6
"cybersam"	3
"David A Stumpf"	3
"fbiville"	2
"Tomaž Bratanič"	2
"Thomas J"	2
"vinay"	1
"deadshot"	1
"Harsh Verma"	1

10 rows
ready to start consuming query after 103 ms, results consumed after another 10 ms

8. Find questions receiving the most comments.

```

MATCH (c:Comment)-[:COMMENTED_ON]->(q:Question)
RETURN q.title AS question,
       count(c) AS comments
ORDER BY comments DESC
LIMIT 10;

```

```
neo4j@neo4j> MATCH (c:Comment)-[:COMMENTED_ON]->(q:Question)
RETURN q.title AS question,
count(c) AS comments
ORDER BY comments DESC
LIMIT 10;
```

question	comments
"neo4j - pagination return same results even when skip and order by are specified"	13
"Spring boot command line application doesn't terminate with neo4j driver"	10
"Neo4j WHERE causes duplicates?"	10
"How do I qualify and name a relationship in Neo4j APOC?"	10
"Related to neo4j and csv files"	10
"Longest path through graph"	9
"AND clause for matching 3 conditions in NE04j"	9
"Neo4j cypher query doesn't return the results as expected into pandas DataFrame"	9
"Connection Unavailable : Trying to connect locally hosted Neo4j from docker app"	9
"Kubernetes pod FileLockException: Lock file has been locked by another process - neo4j 4.2"	9

10 rows
ready to start consuming query after 4 ms, results consumed after another 6 ms

Shortest Path Queries

The shortest path queries look for **connections** between **users, questions, answers, and tags**, instead of **physical/geographical shortest paths**.

1. What is the shortest graph-theoretic path connecting the user **cybersam** and the user **confusedstudent**?

MATCH p = SHORTEST 1

(u1:User {display_name: 'cybersam'})-[*]-(u2:User {display_name: 'confusedstudent'})

RETURN p;

The path could look conceptually like the following:

```
cybersam
|
ASKED
↓
Question
↑
ANSWERED
|
Answer
↑
PROVIDED
|
confusedstudent
```

The **[*]** means that any relationship type and direction can be traversed.

```
neo4j@neo4j> MATCH p = SHORTEST 1 (u1:User {display_name: 'cybersam'})-[*]-(u2:User {display_name: 'confusedstudent'})
RETURN p;
```

```
+-----+
| p
+-----+
|
+-----+
| (:User {display_name: "cybersam", uuid: 974731})-[:PROVIDED]->(:Answer {link: "https://stackoverflow.com/a/65227323", score: 1, is_accepted: FALSE, body_markdown: "Y
our [WITH](https://neo4j.com/docs/cypher-manual/current/clauses/with/) clauses are not specifying all the variables that need to be carried forward to the remainder of
the query. Also, there has at least one typo (a missing comma).
\
|
\ In fact, your query does not even need any `WITH` clauses. Nor does it need to `COLLECT` a list only to immediately `UNWIND` it.
```

2. Find the shortest path from a user to a tag.

MATCH p = SHORTEST 1

(u:User {display_name: 'Aanchal Agarwal'})-[*]-(t:Tag {name: 'python'})

RETURN p;

The result could be

Aanchal Agarwal

|

ASKED

↓

Question

|

TAGGED

↓

python

```
neo4j@neo4j> MATCH p = SHORTEST 1
  (u:User {display_name: 'Aanchal Agarwal'})-[*]-(t:Tag {name: 'python'})
  RETURN p;
+-----+
| p |
+-----+
| (:User {display_name: "Aanchal Agarwal", uid: 10362824})-[:ASKED]->(:Question {body_markdown: "I am working on a project where I need to make GraphQL API with Django Server (Graphene-Django) and Neo4j database. I have looked quite a lot over the internet, but I wasn't able to find any useful resources. If anyone can give a simple example of the flow or suggest some resources, please help.", link: "https://stackoverflow.com/q/67416527", creation_date: 1620297672, answer_count: 2, title: "How can I create GraphQL API with Django and Neo4j Database?", uid: 67416527, view_count: 194})-[:TAGGED]->(:Tag {name: "python", link: "https://stackoverflow.com/questions/tagged/python"}) |
+-----+
1 row
ready to start consuming query after 74 ms, results consumed after another 156 ms
```

3. Find the shortest path between two tags, **python** and **django**.

MATCH p = SHORTEST 1

(t1:Tag {name: 'python'})-[*]-(t2:Tag {name: 'django'})

RETURN p;

Since tags are connected through questions, a path might be:

Python

↑

TAGGED

|

Question

|

TAGGED

↓

django

```

neo4j@neo4j> MATCH p = SHORTEST 1
    (t1:Tag {name: 'python'})-[*]-(t2:Tag {name: 'django'})
RETURN p;
+-----+
| p |
+-----+
| (:Tag {name: "python", link: "https://stackoverflow.com/questions/tagged/python"})<-[TAGGED]-(:Question {accepted_answer_id: 66951367, body_markdown: "I've tried to do a simple project in Django using the Neo4j database. I've installed a 'django-neomodel' library, set the settings as follows:"}) |
+-----+
| \ import os |
| \ from neomodel import config |
| \ |
| \ db_username = os.environ.get('NEO4J_USERNAME') |
| \ |
| \ db_password = os.environ.get('NEO4J_PASSWORD') |
+-----+

```

Path-finding Queries to Solve

1. Write a query that finds the shortest path between two questions whose titles contain **Python** and **Java**? Since this query may take long, bound the search depth to 3.
2. Write a query that finds the shortest path from the user **ASHMIL HUSSAIN K** to a question?
3. Write a query that finds the shortest connection between two tags, **python** and **django**?
4. Write a query that finds the shortest path between **vasant kumar chinnipilli** and **AlTheLazyMonkey** through their StackOverflow activity?

Instead of returning a graphical path, try to print a readable sequence such as:

User: vasant kumar chinnipilli

Question: How do I use Python...

Tag: python

Question: Python performance...

User: AlTheLazyMonkey

HINT: Use RETURN CASE ... WHEN ... WHEN ... ELSE END